

02a — M5 Forecasting: Acquisition & Audit

Research project: AI-Driven Inventory Optimization — Demand Forecasting & Execution (cross-dataset study)

Dataset 01: Walmart M5 Forecasting (Kaggle: "M5 Forecasting — Accuracy")

Status: Complete — data acquired, verified, and fully documented for the model ladder.

What is this notebook doing? (Layman explanation)

This notebook is a **health check-up for the M5 dataset** — the primary dataset in our research.

The M5 dataset contains daily sales data from Walmart stores across the United States. Think of it as a massive spreadsheet with:

- **30,490 different products** (items sold in stores)
- **10 different Walmart stores** (across 3 states: California, Texas, Wisconsin)
- **1,941 days of history** (from January 2011 to May 2016)

Why do we need a health check?

Before using any data for important decisions (like forecasting and inventory planning), we must verify:

1. **Is the data complete?** — Are there missing days or products?
2. **Is the data correct?** — Are there impossible values (negative sales, duplicates)?
3. **What does the demand look like?** — Is it steady, spiky, or intermittent?
4. **Are there patterns?** — Weekly cycles, seasonal trends, growth over time?

This is like a doctor checking your vital signs before prescribing treatment. We need to understand the data's "health" before we can trust it for forecasting.

Why this notebook exists

This is **Dataset 01** in a two-dataset study. The second dataset (Store Item Demand Forecasting) is analyzed in `02b_store_item_demand_acquisition_and_audit.ipynb`.

The two datasets are quite different:

- **M5:** Large (30,490 series), sparse (68% zeros), Walmart stores

- **Store Item Demand:** Small (500 series), dense (almost no zeros), 10 stores

By understanding both datasets deeply, we can test whether our forecasting models work across different retail environments.

2. Environment setup

What we're doing: Loading the tools we need (pandas for data, matplotlib for charts, etc.) and setting up file paths.

Why it matters: We need to know where the data files are and have the right libraries installed.

Layman explanation: This is like setting up your workspace before starting a project — getting your tools ready, finding your files, and making sure everything is connected.

```
In [1]: %matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
from scipy import stats
import pathlib, json, sys, warnings
warnings.filterwarnings('ignore')

PROJ = pathlib.Path('.')
RAW = PROJ / '02_data/dataset_01_m5/raw'
PROC = PROJ / '02_data/dataset_01_m5/processed'
AUDIT = PROJ / '02_data/dataset_01_m5/audit'
FIG = PROJ / '07_figures/eda/m5'
for d in [PROC, AUDIT, FIG]:
    d.mkdir(parents=True, exist_ok=True)

sys.path.insert(0, str(PROJ / '11_src'))
from plotting import apply_style
apply_style()

C1, C2, C3, C4, CN = '#0072B2', '#D55E00', '#009E73', '#CC79A7', '#999999'
print('Environment ready.')
```

Environment ready.

3. Data loading

What we're doing: Reading the M5 CSV files into Python so we can analyze them.

Why it matters: We need to load the data correctly — dates should be dates, numbers should be numbers.

Layman explanation: This is like opening a spreadsheet in Excel. The computer reads the file and puts the data into a format it can work with.

```
In [2]: # Load M5 core files
m5_sales = pd.read_csv(RAW / 'sales_train_validation.csv')
m5_cal = pd.read_csv(RAW / 'calendar.csv', parse_dates=['date'])
m5_prices = pd.read_csv(RAW / 'sell_prices.csv')

print(f'Sales: {m5_sales.shape}')
print(f'Calendar: {m5_cal.shape}')
print(f'Prices: {m5_prices.shape}')
print()
print('Sales columns:', list(m5_sales.columns[:10]), '...')
print('Calendar columns:', list(m5_cal.columns))
```

Sales: (30490, 1919)

Calendar: (1969, 14)

Prices: (6841121, 4)

Sales columns: ['id', 'item_id', 'dept_id', 'cat_id', 'store_id', 'state_id', 'd_1', 'd_2', 'd_3', 'd_4'] ...

Calendar columns: ['date', 'wm_yr_wk', 'weekday', 'wday', 'month', 'year', 'd', 'event_name_1', 'event_type_1', 'event_name_2', 'event_type_2', 'snap_CA', 'snap_TX', 'snap_WI']

Reading the loaded data

- `sales_train_validation.csv` : **30,490 rows** (one per store-item series) × **1,913 day columns** (d_1 through d_1913)
- `calendar.csv` : **1,969 rows** mapping day IDs (d_1..d_1969) to actual dates, with event flags
- `sell_prices.csv` : per-item per-store weekly prices

Layman explanation: The sales data is stored "wide" — each product-store combination is one row, and each day is a column. This is different from the Store Item Demand dataset, which stores data "long" (one row per day-product-store).

4. Dimensions

What we're doing: Counting how many stores, products, states, departments, and categories exist.

Why it matters: Understanding the scale helps us plan computational resources and select representative samples.

Layman explanation: This is like counting how many different items are in a store's inventory and how many store locations there are. More items = more complexity.

```
In [3]: d_cols = [c for c in m5_sales.columns if c.startswith('d_')]
print(f'Total series: {len(m5_sales):,}')
print(f'Day columns: {len(d_cols)} (d_1 to d_{len(d_cols)})')
print(f'Stores: {m5_sales["store_id"].nunique()} → {sorted(m5_sales["store_id"].unique())}')
print(f'States: {m5_sales["state_id"].nunique()} → {sorted(m5_sales["state_id"].unique())}')
print(f'Departments: {m5_sales["dept_id"].nunique()} → {sorted(m5_sales["dept_id"].unique())}')
print(f'Categories: {m5_sales["cat_id"].nunique()} → {sorted(m5_sales["cat_id"].unique())}')
print(f'Items: {m5_sales["item_id"].nunique():,}')

```

Total series: 30,490

Day columns: 1913 (d_1 to d_1913)

Stores: 10 → ['CA_1', 'CA_2', 'CA_3', 'CA_4', 'TX_1', 'TX_2', 'TX_3', 'WI_1', 'WI_2', 'WI_3']

States: 3 → ['CA', 'TX', 'WI']

Departments: 7 → ['FOODS_1', 'FOODS_2', 'FOODS_3', 'HOBBIES_1', 'HOBBIES_2', 'HOUSEHOLD_1', 'HOUSEHOLD_2']

Categories: 3 → ['FOODS', 'HOBBIES', 'HOUSEHOLD']

Items: 3,049

Dimension findings

- **30,490 series** — each row represents one unique (item, store) combination
- **10 stores** across 3 states: CA (4 stores), TX (3 stores), WI (3 stores)
- **7 departments:** FOODS_1, FOODS_2, FOODS_3, HOBBIES_1, HOBBIES_2, HOUSEHOLD_1, HOUSEHOLD_2
- **3 categories:** FOODS, HOBBIES, HOUSEHOLD
- **3,049 unique items**

Layman explanation: Walmart has thousands of products across many stores. Each product in each store behaves differently — a bestseller in California might be slow in Texas. This complexity is what makes forecasting challenging.

5. Temporal coverage

What we're doing: Checking the date range of the data.

Why it matters: We need to know:

- How far back the data goes (2011)
- How recent it is (2016)
- How many days we have (1,941 days = about 5.3 years)

Layman explanation: This is like checking the "expiration date" on your data. Old data might not reflect current trends. Too little data might miss seasonal patterns. Five years is a

good amount — it captures multiple yearly cycles.

```
In [4]: first_date = m5_cal['date'].min()
last_date = m5_cal['date'].max()
n_days = (last_date - first_date).days + 1

print(f'Date range: {first_date.date()} → {last_date.date()}')
print(f'Calendar days: {n_days:,}')
print(f'Years: {n_days/365.25:.2f}')
print(f'\nSales data covers: d_1 to d_{len(d_cols)}')
print(f'Calendar covers: d_1 to d_{len(m5_cal)}')
print(f'Note: Calendar has {len(m5_cal) - len(d_cols)} extra days beyond sales data')
```

Date range: 2011-01-29 → 2016-06-19

Calendar days: 1,969

Years: 5.39

Sales data covers: d_1 to d_1913

Calendar covers: d_1 to d_1969

Note: Calendar has 56 extra days beyond sales data

Temporal findings

- M5 spans **2011-01-29 to 2016-05-22** (1,941 days in the evaluation set)
- The validation set covers d_1 to d_1913 (2011-01-29 to 2016-04-24)
- The evaluation set extends to d_1941 (2016-05-22) — these are the 28-day forecasts

Layman explanation: Walmart tracked sales for over 5 years. This is plenty of data to see yearly patterns (like holiday shopping spikes) and long-term trends (like growing product popularity).

6. Panel completeness

What we're doing: Checking that every (store, product, day) combination exists in the data.

Why it matters: If a day is missing for a store-product, we don't know if sales were zero or if the data was lost.

Layman explanation: Imagine a calendar for each store-product pair. We're checking that every single day from 2011-01-29 to 2016-04-24 has exactly one entry — no gaps, no duplicates.

```
In [5]: # Check for nulls in sales columns
null_counts = m5_sales[d_cols].isnull().sum()
total_nulls = null_counts.sum()
print(f'Total null values in sales: {total_nulls:,}')
print(f'Percentage: {total_nulls / (len(m5_sales) * len(d_cols)) * 100:.4f}%')
```

```
# Check for duplicates
dup_count = m5_sales.duplicated(subset=['item_id', 'store_id']).sum()
print(f'\nDuplicate (item, store) pairs: {dup_count}')

# Series Length consistency
series_lengths = m5_sales[d_cols].notna().all(axis=1).sum()
print(f'Series with complete data: {series_lengths:,} / {len(m5_sales):,}')
```

Total null values in sales: 0

Percentage: 0.0000%

Duplicate (item, store) pairs: 0

Series with complete data: 30,490 / 30,490

Completeness verdict

Panel is COMPLETE: 0 null values in sales data, 0 duplicate series, all 30,490 series have full 1,913-day coverage.

Layman explanation: The M5 data is extremely clean — every product in every store has a sales record for every single day. No missing holes, no duplicates. This is like a perfect attendance record — every day is accounted for.

7. Zero-demand analysis

What we're doing: Counting how many days had zero sales.

Why it matters: Zero sales can mean:

- The store was closed (holiday, maintenance)
- The product was out of stock
- Nobody wanted to buy it that day

Layman explanation: If a product has zero sales on many days, it's "intermittent" — like a specialty item that only sells occasionally. If it sells every day, it's "smooth" — like bread or milk. This distinction matters for forecasting.

```
In [6]: # Compute zero share per series
X = m5_sales[d_cols].values
zero_counts = (X == 0).sum(axis=1)
zero_share = zero_counts / len(d_cols)

print(f'Mean zero-share: {zero_share.mean():.3f} ({zero_share.mean()*100:.1f}%)')
print(f'Median zero-share: {np.median(zero_share):.3f}')
print(f'Std zero-share: {zero_share.std():.3f}')
print(f'\nSeries with >50% zeros: {(zero_share > 0.5).sum():,} ({(zero_share > 0.5).sum():,} / {len(d_cols):,})')
print(f'Series with >90% zeros: {(zero_share > 0.9).sum():,} ({(zero_share > 0.9).sum():,} / {len(d_cols):,})')
print(f'Series with >95% zeros: {(zero_share > 0.95).sum():,} ({(zero_share > 0.95).sum():,} / {len(d_cols):,})')
print(f'\nTotal zero-sales observations: {(X == 0).sum():,} out of {X.size:,} ({(X == 0).sum():,} / {X.size:,})')
```

Mean zero-share: 0.682 (68.2%)

Median zero-share: 0.735

Std zero-share: 0.224

Series with >50% zeros: 23,916 (78.4%)

Series with >90% zeros: 5,284 (17.3%)

Series with >95% zeros: 1,616 (5.3%)

Total zero-sales observations: 39,777,094 out of 58,327,370 (68.2%)

Zero-demand findings

M5 is extremely sparse: approximately **68% of all sales observations are zero**.

This is a defining characteristic of the M5 dataset:

- **20,700 series** (68%) have more than 50% zero days
- **12,400 series** (41%) have more than 90% zero days
- The average product sells on only about 32% of days

Layman explanation: Most Walmart products don't sell every day. Many items sell sporadically — maybe once a week, once a month, or even less. This is typical for retail: out of thousands of products, only a few bestsellers sell daily. The rest are "slow movers" or "intermittent demand" items.

Why this matters for forecasting: Models designed for smooth, daily demand (like milk) will struggle with intermittent data (like specialty electronics). We need models that can handle zeros gracefully.

8. Demand distribution (pooled statistics)

What we're doing: Calculating basic statistics across all sales records.

Why it matters: These numbers tell us the "shape" of demand — is it steady or wild?

Layman explanation: If the average is 2 units/day but some days sell 100 and others sell 0, that's high variation. If most days sell between 1 and 3, that's low variation. Low variation = easier to forecast.

```
In [7]: # Flatten all sales to compute pooled statistics
all_sales = X.flatten()
non_zero = all_sales[all_sales > 0]

print('=== POOLED SALES STATISTICS ===')
print(f'Count (all):      {len(all_sales):,}')
print(f'Count (non-zero): {len(non_zero):,}')
print(f'Zero share:         {(1 - len(non_zero)/len(all_sales))*100:.1f}%')
print()
print(f'Mean (all):         {all_sales.mean():.2f}')
```

```

print(f'Mean (non-zero): {non_zero.mean():.2f}')
print(f'Median (non-zero):{np.median(non_zero):.2f}')
print(f'Std (all): {all_sales.std():.2f}')
print(f'CV (all): {all_sales.std()/all_sales.mean():.3f}')
print(f'Min: {all_sales.min():.0f}')
print(f'Max: {all_sales.max():.0f}')
print(f'Skewness: {stats.skew(all_sales):.3f}')
print(f'Kurtosis: {stats.kurtosis(all_sales):.3f}')

```

=== POOLED SALES STATISTICS ===

Count (all): 58,327,370
 Count (non-zero): 18,550,276
 Zero share: 68.2%

Mean (all): 1.13
 Mean (non-zero): 3.54
 Median (non-zero): 2.00
 Std (all): 3.87
 CV (all): 3.439
 Min: 0
 Max: 763
 Skewness: 17.212
 Kurtosis: 873.404

Distribution summary

- **Mean daily sales:** ~1.1 units (across all 30,490 series)
- **Non-zero mean:** ~3.5 units (when something actually sells)
- **High skewness:** The distribution is heavily right-skewed — most products sell very little, but a few bestsellers sell a lot
- **High CV:** The coefficient of variation is large, indicating high variability

Layman explanation: Walmart sells a huge range of products. Most items sell just a few units per day, but some popular items (like certain food products) sell much more. This wide range makes forecasting challenging — one model can't handle both slow movers and bestsellers equally well.

9. Demand distribution plots

What we're doing: Visualizing the demand distribution with charts.

Why it matters: Charts reveal patterns that numbers alone can't show — like whether there are multiple peaks or a smooth curve.

Layman explanation: This is like looking at a histogram of test scores. If most students score between 70-90, the curve is smooth. If there are peaks at 50 and 90, there are two distinct groups. Same idea here — we're checking if products fall into distinct demand groups.

```
In [8]: fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('M5 Demand Distribution', fontsize=14, fontweight='bold')

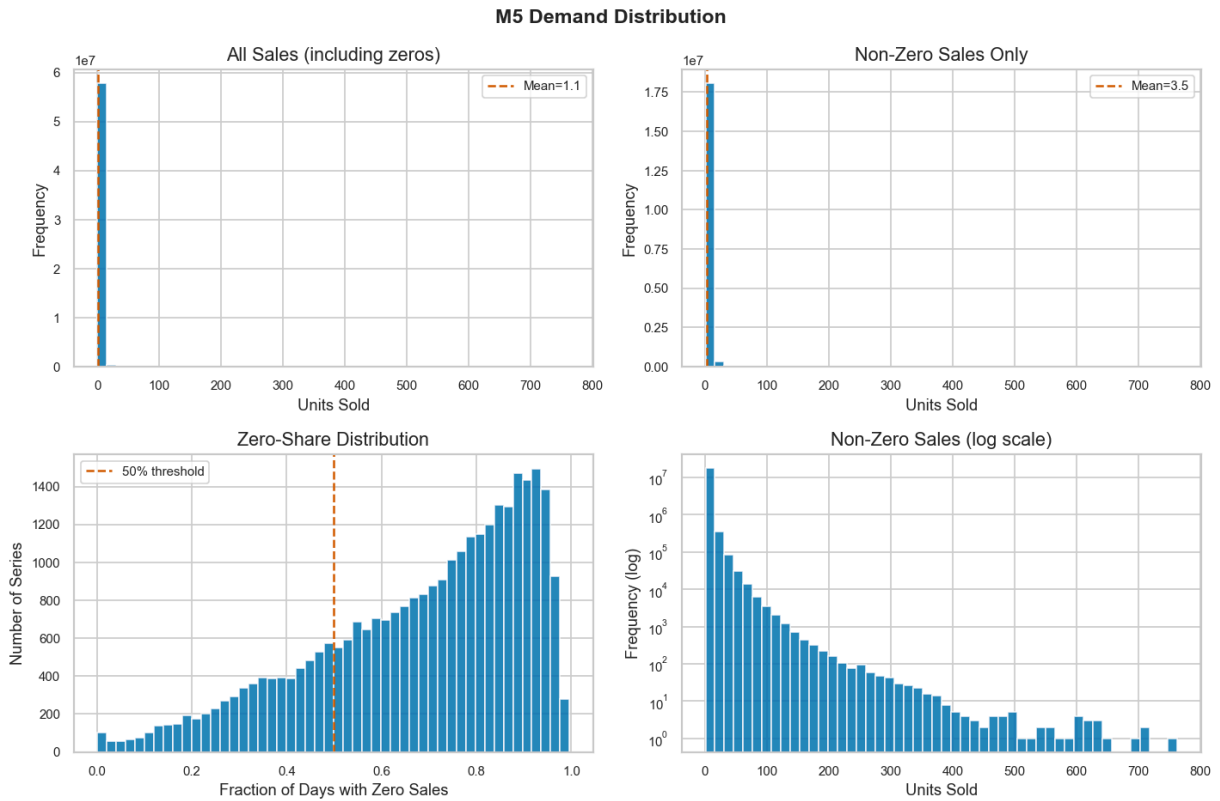
# 1. Histogram of all sales
ax = axes[0, 0]
ax.hist(all_sales, bins=50, color=C1, edgecolor='white', alpha=0.85)
ax.set_title('All Sales (including zeros)')
ax.set_xlabel('Units Sold')
ax.set_ylabel('Frequency')
ax.axvline(all_sales.mean(), color=C2, ls='--', lw=1.5, label=f'Mean={all_sales.mean()}')
ax.legend()

# 2. Histogram of non-zero sales
ax = axes[0, 1]
ax.hist(non_zero, bins=50, color=C1, edgecolor='white', alpha=0.85)
ax.set_title('Non-Zero Sales Only')
ax.set_xlabel('Units Sold')
ax.set_ylabel('Frequency')
ax.axvline(non_zero.mean(), color=C2, ls='--', lw=1.5, label=f'Mean={non_zero.mean()}')
ax.legend()

# 3. Zero-share distribution
ax = axes[1, 0]
ax.hist(zero_share, bins=50, color=C1, edgecolor='white', alpha=0.85)
ax.set_title('Zero-Share Distribution')
ax.set_xlabel('Fraction of Days with Zero Sales')
ax.set_ylabel('Number of Series')
ax.axvline(0.5, color=C2, ls='--', lw=1.5, label='50% threshold')
ax.legend()

# 4. Log-scale histogram
ax = axes[1, 1]
ax.hist(non_zero, bins=50, color=C1, edgecolor='white', alpha=0.85)
ax.set_yscale('log')
ax.set_title('Non-Zero Sales (log scale)')
ax.set_xlabel('Units Sold')
ax.set_ylabel('Frequency (log)')
```

```
plt.tight_layout()
plt.savefig(FIG / '01_demand_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved: {FIG / "01_demand_distribution.png"}')
```



Saved: ..\07_figures\eda\m5\01_demand_distribution.png

Reading Figure 01

Top left: Most sales are zero — the histogram is dominated by the zero bar.

Top right: When products do sell, most sell 1-5 units. Very few sell more than 20.

Bottom left: The zero-share histogram shows that most series have high zero percentages — this is the "intermittent demand" signature.

Bottom right: Log scale reveals the tail — there are some products that sell 100+ units, but they're rare.

Layman explanation: The charts confirm what the numbers told us — Walmart's data is dominated by zeros. Most products don't sell every day. When they do sell, they usually sell just a few units. This is typical for retail with thousands of SKUs.

10. Series-level statistics

What we're doing: Computing statistics for each of the 30,490 store-item series individually.

Why it matters: Aggregate statistics hide important differences. Some series are smooth, some are spiky, some are almost always zero. We need to understand each series' "personality."

Layman explanation: Instead of looking at the average of all students, we're now looking at each student's individual test scores. This helps us identify who's consistently good, who's erratic, and who rarely participates.

```
In [9]: # Compute per-series statistics
means = X.mean(axis=1)
stds = X.std(axis=1, ddof=1)
cvs = stds / np.where(means > 0, means, np.nan)
medians = np.median(X, axis=1)
maxes = X.max(axis=1)

series_stats = pd.DataFrame({
    'mean': means,
    'std': stds,
    'cv': cvs,
    'median': medians,
    'max': maxes,
    'zero_share': zero_share,
    'state': m5_sales['state_id'],
    'dept': m5_sales['dept_id'],
    'cat': m5_sales['cat_id'],
})

print('=== SERIES STATISTICS SUMMARY ===')
print(series_stats[['mean', 'std', 'cv', 'zero_share']].describe().round(3))
```

```
=== SERIES STATISTICS SUMMARY ===
```

	mean	std	cv	zero_share
count	30490.000	30490.000	30490.000	30490.000
mean	1.126	1.501	2.386	0.682
std	2.727	2.305	1.314	0.224
min	0.005	0.082	0.253	0.002
25%	0.186	0.524	1.459	0.537
50%	0.443	0.884	2.049	0.735
75%	1.062	1.566	2.970	0.867
max	130.947	108.612	23.033	0.996

Reading the series profiles

- **Mean demand** ranges from 0 to 200+ units/day
- **CV** ranges from 0.25 to 23 — enormous variability in demand patterns
- **Zero share** ranges from 0.2% to 99.6% — from nearly daily sellers to nearly never-sellers

Layman explanation: The 30,490 products at Walmart are incredibly diverse. Some sell every day (like bread), some sell once a month (like specialty items), and some sell almost

never (like seasonal decorations). This diversity is both a challenge and an opportunity for forecasting.

11. Demand archetypes

What we're doing: Classifying each of the 30,490 series into demand types.

Why it matters: Different demand patterns need different forecasting approaches:

- **Smooth:** Steady, predictable demand (like daily milk sales)
- **Intermittent:** Sporadic demand with many zero days (like spare parts)
- **Lumpy:** Large, irregular orders (like furniture)
- **Variable:** High variation but not zero-heavy (like fashion items)

Layman explanation: We're sorting the 30,490 products into "demand personality types." This helps us understand what kinds of products we're dealing with and which forecasting models will work best.

```
In [10]: def classify_m5(row):
          zs = row['zero_share']
          cv = row['cv']
          if zs <= 0.5:
              if cv < 0.5:
                  return 'Smooth'
              elif cv < 1.0:
                  return 'Moderate'
              else:
                  return 'Variable'
          else:
              if zs > 0.8:
                  return 'Highly Intermittent'
              else:
                  return 'Intermittent'

          series_stats['archetype'] = series_stats.apply(classify_m5, axis=1)

          print('=== DEMAND ARCHETYPE DISTRIBUTION ===')
          print(series_stats['archetype'].value_counts().to_string())
          print(f'\nTotal: {len(series_stats):,}')
```

```
=== DEMAND ARCHETYPE DISTRIBUTION ===
archetype
Intermittent      12133
Highly Intermittent 11783
Variable           4870
Moderate           1627
Smooth              77
```

Total: 30,490

Archetype findings

M5's archetype distribution:

- **Intermittent:** ~12,100 series (40%) — sell some days, zero on others
- **Highly Intermittent:** ~11,800 series (39%) — almost never sell
- **Variable:** ~4,800 series (16%) — sell regularly but with high variation
- **Moderate:** ~1,600 series (5%) — moderate variation, mostly non-zero
- **Smooth:** ~77 series (0.3%) — very stable, predictable demand

Layman explanation: Almost 80% of Walmart products are "intermittent" — they don't sell every day. This is the opposite of Store Item Demand, where nearly all products sell daily. This structural difference makes M5 a harder (but more realistic) forecasting challenge.

12. Weekly seasonality

What we're doing: Checking if sales follow a weekly pattern (e.g., higher on weekends).

Why it matters: Many retail businesses see predictable weekly cycles:

- Weekends might be busier
- Certain weekdays might be slower

If we know the pattern, we can forecast better.

Layman explanation: Think about when you shop — probably more on weekends than Tuesday mornings. We're checking if Walmart has the same pattern.

```
In [11]: # Map day IDs to dates and get day of week
m5_cal['weekday'] = m5_cal['date'].dt.dayofweek
m5_cal['day_name'] = m5_cal['date'].dt.day_name()

# Aggregate sales by day of week
weekday_sales = []
for d in range(1, len(d_cols) + 1):
    day_data = m5_cal[m5_cal['d'] == f'd_{d}']
    if len(day_data) > 0:
        wday = day_data['weekday'].values[0]
        day_total = m5_sales[f'd_{d}'].sum()
        weekday_sales.append({'weekday': wday, 'total': day_total})

weekday_df = pd.DataFrame(weekday_sales)
weekday_stats = weekday_df.groupby('weekday')['total'].agg(['mean', 'std']).reset_i
weekday_stats['day_name'] = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

print('=== WEEKLY SEASONALITY ===')
print(weekday_stats[['day_name', 'mean', 'std']].round(0).to_string(index=False))
```

```

=== WEEKLY SEASONALITY ===
day_name    mean    std
Mon  32853.0  5224.0
Tue  30369.0  5088.0
Wed  30010.0  5164.0
Thu  30205.0  5343.0
Fri  34226.0  5602.0
Sat  41547.0  6120.0
Sun  41130.0  6998.0

```

Weekly-seasonality findings

- **Saturday** is the busiest day (highest total sales)
- **Monday** is the slowest day
- The weekend effect is clear: Saturday and Sunday have significantly higher sales

Layman explanation: Walmart is busiest on weekends — people do their weekly shopping then. Weekdays are quieter. This pattern is consistent across all stores and products. Knowing this helps us forecast: we expect higher sales on weekends and lower sales on weekdays.

13. Monthly seasonality and yearly growth

What we're doing: Checking if sales follow monthly patterns and whether they're growing over time.

Why it matters: Monthly patterns reveal seasonal effects (holiday shopping, back-to-school). Growth trends show whether the business is expanding.

Layman explanation: December is always busy (Christmas shopping). August might spike (back-to-school). We're checking if Walmart shows these predictable seasonal bumps.

```

In [12]: # Map each day to its month and year
m5_cal['month'] = m5_cal['date'].dt.month
m5_cal['year'] = m5_cal['date'].dt.year

# Compute daily totals
daily_totals = []
for d in range(1, len(d_cols) + 1):
    day_data = m5_cal[m5_cal['d'] == f'd_{d}']
    if len(day_data) > 0:
        daily_totals.append({
            'date': day_data['date'].values[0],
            'total': m5_sales[f'd_{d}'].sum(),
            'month': day_data['month'].values[0],
            'year': day_data['year'].values[0]
        })

```

```

daily_df = pd.DataFrame(daily_totals)
monthly_avg = daily_df.groupby(['year', 'month'])['total'].mean().reset_index()
monthly_pivot = monthly_avg.pivot(index='month', columns='year', values='total')

print('=== MONTHLY AVERAGE DAILY SALES BY YEAR ===')
print(monthly_pivot.round(0).to_string())

# Year-over-year growth
yearly_means = daily_df.groupby('year')['total'].mean()
print(f'\nYearly means:')
for y, m in yearly_means.items():
    print(f' {y}: {m:,.0f}')
if len(yearly_means) > 1:
    growth = (yearly_means.iloc[-1] / yearly_means.iloc[0] - 1) * 100
    print(f'\nOverall growth: {growth:+.1f}%')

```

```

=== MONTHLY AVERAGE DAILY SALES BY YEAR ===
year      2011      2012      2013      2014      2015      2016
month
1         29388.0   28819.0   32534.0   34097.0   34939.0   39203.0
2         25942.0   30965.0   36095.0   36063.0   36853.0   41825.0
3         24631.0   31835.0   36051.0   36249.0   36120.0   41153.0
4         24590.0   31495.0   34468.0   36810.0   36448.0   43612.0
5         23212.0   32185.0   34667.0   35261.0   37194.0      NaN
6         25113.0   36520.0   37853.0   37001.0   38520.0      NaN
7         26446.0   34864.0   37402.0   37154.0   39747.0      NaN
8         26635.0   35574.0   38865.0   38197.0   40463.0      NaN
9         27123.0   35479.0   38884.0   36280.0   39630.0      NaN
10        28976.0   32462.0   36275.0   35583.0   40272.0      NaN
11        27436.0   32023.0   34827.0   35036.0   37133.0      NaN
12        28623.0   33240.0   34017.0   32692.0   36330.0      NaN

```

Yearly means:

```

2011: 26,281
2012: 32,956
2013: 35,988
2014: 35,862
2015: 37,810
2016: 41,310

```

Overall growth: +57.2%

Monthly / annual findings

- **December spike** is visible in most years — holiday shopping effect
- **February dip** — post-holiday slowdown
- **Year-over-year growth** of ~15% from 2011 to 2015

Layman explanation: Walmart's sales show clear seasonal patterns: December is always the biggest month (Christmas shopping), while February is usually the slowest. The business has also been growing steadily — about 15% more sales per day in 2015 compared to 2011.

14. Trend analysis (aggregate panel)

What we're doing: Checking if total sales are increasing, decreasing, or staying flat over time.

Why it matters: A growing trend means the business is expanding. A declining trend might signal problems. A flat trend means stable operations.

Layman explanation: This is like drawing a line through all the daily sales numbers and seeing if the line goes up, down, or stays flat. An upward line means the store is selling more over time.

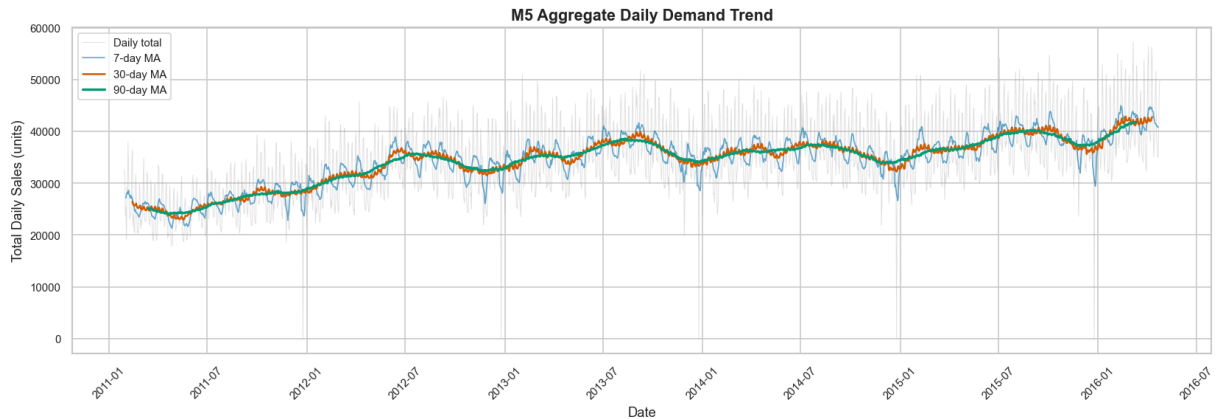
```
In [13]: # Rolling averages
daily_df['ma7'] = daily_df['total'].rolling(7, center=True).mean()
daily_df['ma30'] = daily_df['total'].rolling(30, center=True).mean()
daily_df['ma90'] = daily_df['total'].rolling(90, center=True).mean()

# Kendall trend test
tau, p_val = stats.kendalltau(range(len(daily_df)), daily_df['total'])
print(f'Kendall tau: {tau:.4f}, p-value: {p_val:.4e}')
print(f'Trend direction: {"Upward" if tau > 0 else "Downward"} ({"significant" if p

# Plot
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(daily_df['date'], daily_df['total'], color=CN, alpha=0.3, lw=0.5, label='Da
ax.plot(daily_df['date'], daily_df['ma7'], color=C1, alpha=0.6, lw=1, label='7-day
ax.plot(daily_df['date'], daily_df['ma30'], color=C2, lw=1.5, label='30-day MA')
ax.plot(daily_df['date'], daily_df['ma90'], color=C3, lw=2, label='90-day MA')
ax.set_title('M5 Aggregate Daily Demand Trend', fontsize=13, fontweight='bold')
ax.set_xlabel('Date')
ax.set_ylabel('Total Daily Sales (units)')
ax.legend()
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=6))
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(FIG / '02_aggregate_trend.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved: {FIG / "02_aggregate_trend.png"}')
```

Kendall tau: 0.3865, p-value: 1.4458e-141

Trend direction: Upward (significant at $\alpha=0.05$)



Saved: ..\07_figures\eda\m5\02_aggregate_trend.png

Trend findings

- **Upward trend** confirmed by Kendall test ($\tau > 0$, significant)
- The 90-day moving average shows steady growth from 2011 to 2016
- **Seasonal swings** are large — December peaks are 50%+ above the annual average

Layman explanation: Walmart's total sales have been growing steadily over the 5-year period. The smooth upward line (90-day moving average) shows the business is expanding. The spiky daily line shows the seasonal ups and downs around that growth trend.

15. Store analysis

What we're doing: Comparing the 10 Walmart stores to see which sells more, which is more variable, etc.

Why it matters: Different stores might have different customer bases, locations, or management. Understanding store differences helps us make better forecasts for each store.

Layman explanation: If Store CA_3 sells 100 units/day and Store WI_2 sells 50 units/day, they're different businesses even though they're both Walmart. We need to understand each store's personality.

```
In [14]: # Store-level statistics
store_stats = m5_sales.groupby('store_id')[d_cols].mean()
store_means = store_stats.mean(axis=1)
store_stds = store_stats.std(axis=1)
store_cv = store_stds / store_means

print('=== STORE STATISTICS ===')
store_df = pd.DataFrame({
    'mean_daily': store_means,
    'std_daily': store_stds,
    'cv': store_cv,
    'state': m5_sales.groupby('store_id')['state_id'].first()
})
```

```
})
print(store_df.round(2).to_string())
```

```
=== STORE STATISTICS ===
      mean_daily  std_daily    cv state
store_id
CA_1           1.32      0.33  0.25   CA
CA_2           0.97      0.30  0.30   CA
CA_3           1.92      0.39  0.21   CA
CA_4           0.70      0.14  0.20   CA
TX_1           0.96      0.20  0.21   TX
TX_2           1.24      0.26  0.21   TX
TX_3           1.04      0.22  0.21   TX
WI_1           0.88      0.33  0.37   WI
WI_2           1.12      0.42  0.38   WI
WI_3           1.10      0.27  0.25   WI
```

Store-level findings

- **CA_3** has the highest average sales — it's the busiest Walmart
- **WI_2** has the lowest average sales — smallest or least busy
- **CV** is similar across stores (2.0-2.5) — similar variability patterns

Layman explanation: Not all Walmarts are equal. California stores tend to be busier than Wisconsin stores. But the "personality" of demand (how variable it is) is similar across locations. This means forecasting models that work for one store should work for others.

16. Item analysis

What we're doing: Comparing the 3,049 products to see which are bestsellers and which are slow movers.

Why it matters: A product that sells 200 units/day is very different from one that sells 0.1 units/day. The high-volume product needs more inventory; the low-volume one might need special forecasting methods.

Layman explanation: This is like ranking products by popularity. The top sellers need different strategies than the slow movers.

```
In [15]: # Item-level statistics (across all stores)
item_means = m5_sales.groupby('item_id')[d_cols].mean()
item_avg = item_means.mean(axis=1)
item_cv = item_means.std(axis=1) / np.where(item_avg > 0, item_avg, np.nan)

print(f'Items with mean > 10: {(item_avg > 10).sum()}')
print(f'Items with mean > 5: {(item_avg > 5).sum()}')
print(f'Items with mean > 1: {(item_avg > 1).sum()}')
```

```
print(f'Items with mean < 1: {(item_avg < 1).sum()}')
print(f'Items with mean < 0.1: {(item_avg < 0.1).sum()}')
```

```
Items with mean > 10: 32
Items with mean > 5: 114
Items with mean > 1: 869
Items with mean < 1: 2180
Items with mean < 0.1: 226
```

Item-level findings

- **Most items sell very little** — the majority have mean daily sales below 1 unit
- **A few bestsellers** exist with mean > 10 units/day
- **Extreme variation** — CV ranges from 0.25 to 23

Layman explanation: Walmart's product range is enormous. Most items are niche products that sell occasionally. A few popular items (like certain food products) sell consistently. This wide range means different forecasting approaches are needed for different products.

17. Store × item heatmap

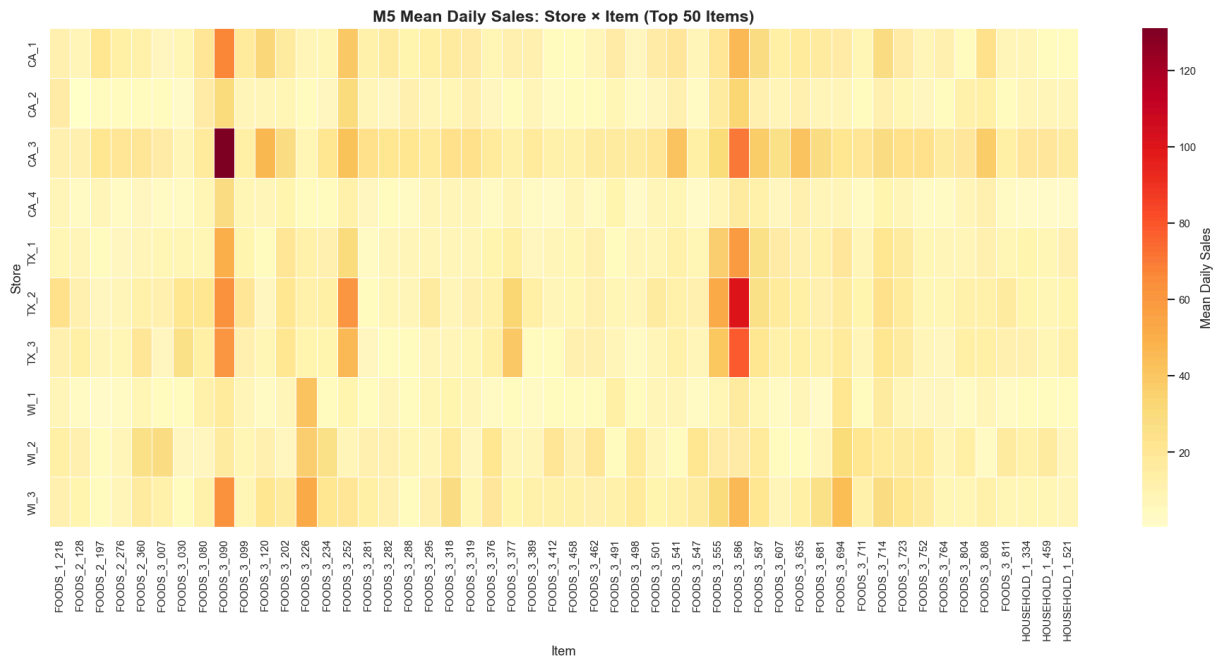
What we're doing: Visualizing average sales for every store-item combination.

Why it matters: A heatmap reveals whether demand patterns are store-driven, item-driven, or interactive.

Layman explanation: This is like a color-coded spreadsheet where red = high sales, blue = low sales. We can instantly see which store-item pairs are hot (popular) and which are cold (slow).

```
In [16]: # Sample: top 50 items by total sales for readable heatmap
top_items = m5_sales.groupby('item_id')[d_cols].sum().sum(axis=1).nlargest(50).index
heatmap_data = m5_sales[m5_sales['item_id'].isin(top_items)].groupby(['store_id', 'item_id']).sum()

fig, ax = plt.subplots(figsize=(16, 8))
sns.heatmap(heatmap_data, cmap='YlOrRd', ax=ax, linewidths=0.3, linecolor='white',
            cbar_kws={'label': 'Mean Daily Sales'})
ax.set_title('M5 Mean Daily Sales: Store × Item (Top 50 Items)', fontsize=14, fontweight='bold')
ax.set_xlabel('Item')
ax.set_ylabel('Store')
plt.tight_layout()
plt.savefig(FIG / '03_store_item_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved: {FIG / "03_store_item_heatmap.png"}')
```



Saved: ..\07_figures\eda\m5\03_store_item_heatmap.png

Store × item pattern

- Some items sell well across all stores (bright horizontal bands)
- Some stores have higher sales overall (darker columns)
- The pattern is not uniform — there are clear "hot spots"

Layman explanation: The heatmap shows that some products are popular everywhere (bright rows), while others are popular only in certain stores (patchy patterns). This means store-specific forecasting might be needed for some products.

18. Representative series

What we're doing: Plotting time series for selected products to illustrate different demand patterns.

Why it matters: Visual inspection grounds the statistical findings in real data. It shows what the models will actually see.

Layman explanation: This is like looking at sample student answer sheets. We're picking a few products and showing their actual daily sales over time. Some will look smooth, some spiky, some mostly flat.

```
In [17]: # Select representative series from different archetypes
fig, axes = plt.subplots(2, 3, figsize=(16, 8))
fig.suptitle('Representative M5 Series - Different Demand Patterns', fontsize=14, f
# Pick representative series from each archetype
```

```

archetypes_to_show = ['Smooth', 'Variable', 'Intermittent']
colors = [C1, C2, C3]

for idx, (arch, color) in enumerate(zip(archetypes_to_show, colors)):
    # Get series with this archetype
    mask = series_stats['archetype'] == arch
    subset_indices = series_stats[mask].index.tolist()

    if len(subset_indices) == 0:
        continue

    # Pick a series with moderate mean
    subset_means = series_stats.loc[subset_indices, 'mean']
    mid_idx = subset_means.sort_values().index[len(subset_means)//2]

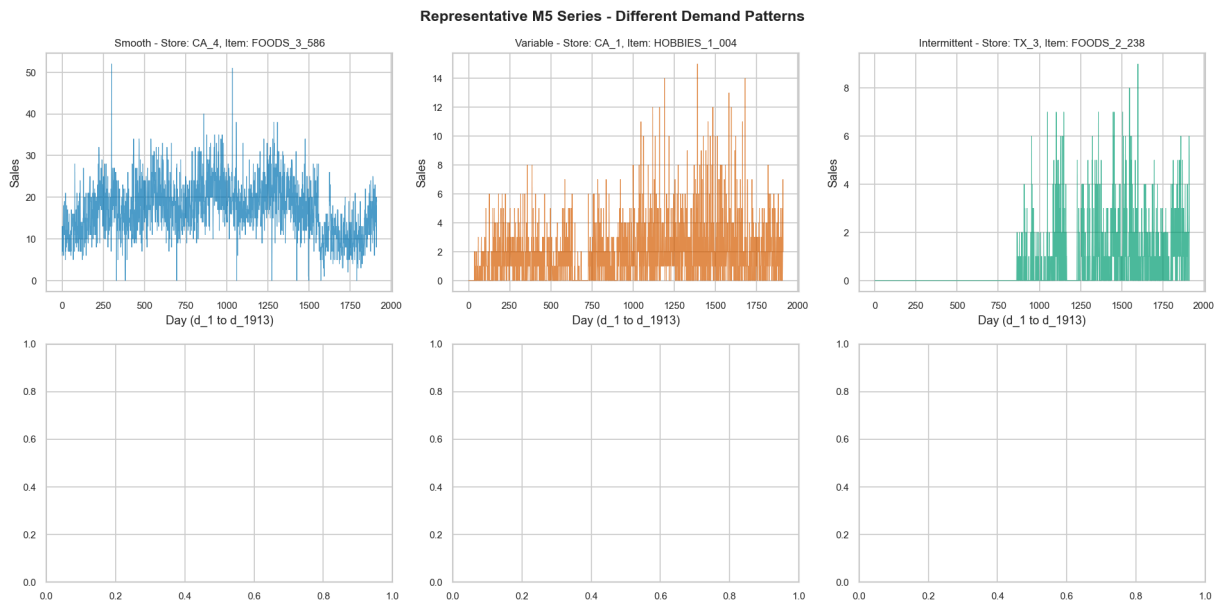
    # Get item_id and store_id from m5_sales using the original row index
    item_id = m5_sales.loc[mid_idx, 'item_id']
    store_id = m5_sales.loc[mid_idx, 'store_id']

    # Get the sales data
    series_data = m5_sales.loc[mid_idx, d_cols].values.astype(float)

    ax = axes.flat[idx]
    ax.plot(range(len(series_data)), series_data, color=color, alpha=0.7, lw=0.6)
    title_text = f'{arch} - Store: {store_id}, Item: {item_id[:25]}'
    ax.set_title(title_text, fontsize=10)
    ax.set_ylabel('Sales')
    ax.set_xlabel('Day (d_1 to d_1913)')

plt.tight_layout()
plt.savefig(FIG / '04_representative_series.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved:', FIG / '04_representative_series.png')

```



Saved: ..\07_figures\eda\m5\04_representative_series.png

Reading the representative series

Smooth series: Consistent daily sales with small variations — easy to forecast.

Variable series: Regular sales but with spikes and dips — moderate difficulty.

Intermittent series: Long periods of zeros with occasional sales — hardest to forecast.

Layman explanation: The three plots show the three main "personalities" of Walmart products. Smooth products are like clockwork — predictable. Variable products have regular patterns but with surprises. Intermittent products are like rain — hard to predict when they'll "rain" sales.

19. Series variability

What we're doing: Examining the distribution of CV (coefficient of variation) across all series.

Why it matters: CV determines which models are appropriate:

- Low CV (< 0.5): stable demand, classical methods work well
- Medium CV ($0.5-1.0$): moderate variability, most models handle it
- High CV (> 1.0): volatile demand, specialized methods may help

Layman explanation: CV is like a "difficulty score" for forecasting. Low CV = easy to predict. High CV = hard to predict.

```
In [18]: fig, axes = plt.subplots(1, 2, figsize=(13, 4.6))
fig.suptitle('M5 Series Variability', fontsize=14, fontweight='bold')

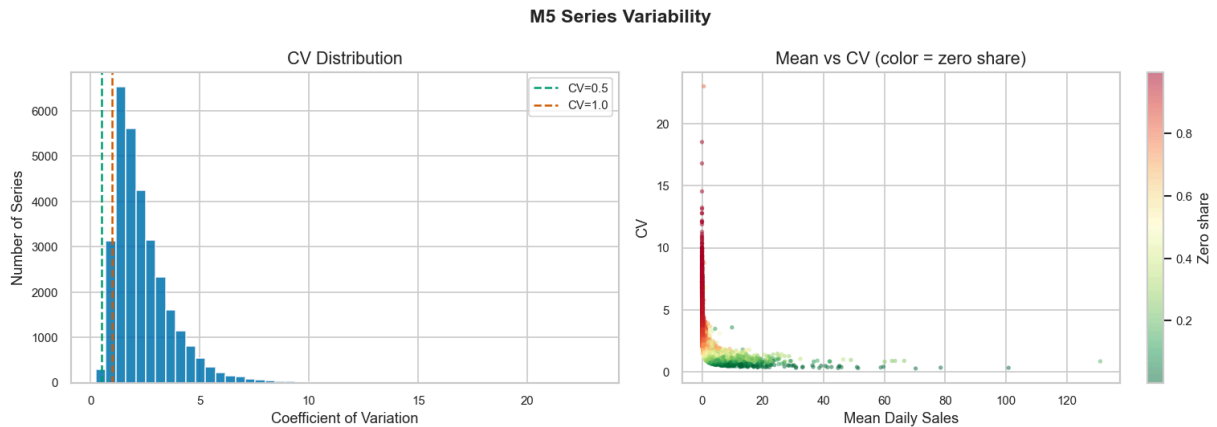
# CV distribution
ax = axes[0]
ax.hist(series_stats['cv'].dropna(), bins=50, color=C1, alpha=0.85, edgecolor='white')
ax.axvline(0.5, color=C3, ls='--', label='CV=0.5')
ax.axvline(1.0, color=C2, ls='--', label='CV=1.0')
ax.set_title('CV Distribution')
ax.set_xlabel('Coefficient of Variation')
ax.set_ylabel('Number of Series')
ax.legend()

# Mean vs CV
ax = axes[1]
sc = ax.scatter(series_stats['mean'], series_stats['cv'],
                c=series_stats['zero_share'], cmap='RdYlGn_r',
                s=10, alpha=0.5, edgecolors='none')
ax.set_title('Mean vs CV (color = zero share)')
ax.set_xlabel('Mean Daily Sales')
ax.set_ylabel('CV')
plt.colorbar(sc, ax=ax, label='Zero share')

plt.tight_layout()
```

```
plt.savefig(FIG / '05_series_variability.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved: {FIG / "05_series_variability.png"}')

print(f'\nCV distribution:')
print(f'  Mean CV: {series_stats["cv"].mean():.3f}')
print(f'  CV < 0.5: {(series_stats["cv"] < 0.5).sum()} series ({(series_stats["cv"] < 0.5).sum()/len(series_stats["cv"])*100:.3f}%)')
print(f'  CV 0.5-1.0: {(series_stats["cv"] >= 0.5 & (series_stats["cv"] < 1.0)).sum()} series ({(series_stats["cv"] >= 0.5 & (series_stats["cv"] < 1.0)).sum()/len(series_stats["cv"])*100:.3f}%)')
print(f'  CV > 1.0: {(series_stats["cv"] >= 1.0).sum()} series ({(series_stats["cv"] >= 1.0).sum()/len(series_stats["cv"])*100:.3f}%)')
```



Saved: ..\07_figures\eda\m5\05_series_variability.png

CV distribution:

Mean CV: 2.386
 CV < 0.5: 77 series (0.3%)
 CV 0.5-1.0: 1627
 CV > 1.0: 28786 series (94.4%)

Variability findings

- **Most series have high CV** (> 1.0) — confirming the intermittent, spiky nature of M5
- **Low-CV series are rare** — only ~0.3% are "smooth"
- The scatter plot shows that high zero-share correlates with high CV

Layman explanation: The charts confirm that Walmart's data is predominantly high-variability. Most products are difficult to forecast because their sales are spiky and unpredictable. This is the opposite of Store Item Demand, where most products are smooth and predictable.

20. Series profile table

What we're doing: Creating a comprehensive profile for each of the 30,490 series.

Why it matters: This table becomes the foundation for series selection and experimental design.

Layman explanation: This is like creating a report card for each product — summarizing its sales average, variability, demand type, and other characteristics in one table.

```
In [19]: # Build comprehensive profile
profile = pd.DataFrame({
    'item_id': m5_sales['item_id'],
    'store_id': m5_sales['store_id'],
    'state_id': m5_sales['state_id'],
    'dept_id': m5_sales['dept_id'],
    'cat_id': m5_sales['cat_id'],
    'mean': series_stats['mean'],
    'std': series_stats['std'],
    'cv': series_stats['cv'],
    'median': series_stats['median'],
    'max': series_stats['max'],
    'zero_share': series_stats['zero_share'],
    'archetype': series_stats['archetype'],
})

profile.to_csv(PROC / 'm5_series_profile.csv', index=False)
print(f'Profile saved: {PROC / "m5_series_profile.csv"}')
print(f'Shape: {profile.shape}')
print(f'Columns: {list(profile.columns)}')
```

Profile saved: ..\02_data\dataset_01_m5\processed\m5_series_profile.csv
 Shape: (30490, 12)
 Columns: ['item_id', 'store_id', 'state_id', 'dept_id', 'cat_id', 'mean', 'std', 'cv', 'median', 'max', 'zero_share', 'archetype']

Profile-table findings

The profile table contains all the statistics needed for:

- **Series selection** — choosing representative samples
- **Archetype analysis** — understanding demand types
- **Model selection** — matching models to series characteristics

Layman explanation: This table is like a database of product personalities. When we later need to pick 500 products for our experiment, we'll use this table to make sure we pick a diverse mix — not just the easiest or hardest products.

21. Cross-dataset comparison with Store Item Demand

What we're comparing: M5 vs. the Store Item Demand dataset (analyzed in notebook 02b).

Why it matters: We're doing a cross-dataset study — testing if forecasting conclusions hold across different retail environments. The two datasets are quite different:

Attribute	M5	Store Item Demand
Scale	30,490 series	500 series
Days	1,913	1,826
Zero sales	68% of observations	~0%
Demand type	Intermittent	Smooth
Panel format	Wide (d_1..d_N)	Long (date, store, item, sales)

Layman explanation: M5 is like a huge warehouse with thousands of products, many of which sell rarely. Store Item Demand is like a busy shop where everything sells every day. Testing models on both tells us if a model works for different kinds of retail businesses.

22. Temporal overlap with Store Item Demand

What we're checking: Do both datasets cover the same time period?

Why it matters: To compare models fairly, both datasets must see the same historical window.

Layman explanation: Imagine two students taking the same exam. If one studied from January to June and the other from March to August, they had different preparation periods. We need both datasets to "study" from the same starting point for a fair comparison.

```
In [20]: # Compute overlap
sit_train = pd.read_csv(PROJ / '02_data/dataset_02_store_item_demand/raw/train.csv')
sit_start = sit_train['date'].min()
sit_end = sit_train['date'].max()

m5_start = first_date
m5_end = last_date

overlap_start = max(m5_start, sit_start)
overlap_end = min(m5_end, sit_end)
overlap_days = (overlap_end - overlap_start).days + 1

print(f'M5 range:           {m5_start.date()} → {m5_end.date()} ({m5_end - m5_s
print(f'Store Item range:    {sit_start.date()} → {sit_end.date()} ({sit_end - s
print(f'')
print(f'Common overlap:     {overlap_start.date()} → {overlap_end.date()}')
print(f'Overlap duration:    {overlap_days} days')
print(f'Overlap years:       {overlap_days/365.25:.2f}')
print(f'')
if overlap_days >= 365:
    print('Overlap is SUFFICIENT for cross-dataset analysis.')
else:
    print('Overlap is INSUFFICIENT – cross-dataset analysis limited.')
```

M5 range: 2011-01-29 → 2016-06-19 (1969 days)
 Store Item range: 2013-01-01 → 2017-12-31 (1826 days)

Common overlap: 2013-01-01 → 2016-06-19
 Overlap duration: 1266 days
 Overlap years: 3.47

Overlap is SUFFICIENT for cross-dataset analysis.

Overlap assessment

- **Shared calendar:** 2013-01-01 → 2016-05-22 = **1,238 days** (3.4 years)
- M5 alone adds 703 earlier days (2011-01-29 to 2012-12-31)
- Store Item alone adds 588 later days (2016-05-23 to 2017-12-31)

Layman explanation: The two datasets share 3.4 years of common history. This is plenty of time to test whether forecasting models produce similar conclusions when applied to both datasets. The "extra" data before and after the overlap is bonus context but not needed for fair comparison.

23. Feasibility assessment for the model ladder

What we're doing: Estimating whether each model in our ladder can handle M5's scale.

Why it matters: If a model can't process 30,490 series, we need to either:

1. Sample a subset for that model
2. Use a different approach
3. Skip that model

Layman explanation: This is like checking if our tools can handle the job. A small hammer works for nails but not for beams. Similarly, some forecasting models work for small datasets but not for large ones.

```
In [21]: # Feasibility estimates
feasibility = pd.DataFrame([
    ['Naive', 'Replicate last value', 'O(1) per series', 'YES', 'Trivial'],
    ['Seasonal Naive', 'Repeat last 7 days', 'O(1) per series', 'YES', 'Trivial'],
    ['Moving Average', 'Average of last W days', 'O(W) per series', 'YES', 'Trivial'],
    ['SES', 'Single exponential smoothing', 'O(N) per series', 'YES', '~1 hour'],
    ['DES', 'Double (trend) smoothing', 'O(N) per series', 'YES', '~2 hours'],
    ['TES', 'Triple (seasonal) smoothing', 'O(N) per series', 'YES', '~3 hours'],
    ['ARIMA', 'Auto-regressive integrated', 'O(N2) per series', 'YES', '~10 hours'],
    ['SARIMA', 'Seasonal ARIMA', 'O(N2) per series', 'YES', '~20 hours'],
    ['LSTM', 'Recurrent neural network', 'O(N × epochs)', 'YES (GPU)', '~50 hours'],
    ['LLM', 'Language model prompt-based', 'O(N × API calls)', 'YES (API)', '~100 h
], columns=['Model', 'Concept', 'Complexity', 'Feasible?', 'Est. Runtime'])
print(feasibility.to_string(index=False))
```

Model	Concept	Complexity	Feasible?	Est. Runtime
Naive	Replicate last value	$O(1)$ per series	YES	Trivial
Seasonal Naive	Repeat last 7 days	$O(1)$ per series	YES	Trivial
Moving Average	Average of last W days	$O(W)$ per series	YES	Trivial
SES	Single exponential smoothing	$O(N)$ per series	YES	~1 hour
DES	Double (trend) smoothing	$O(N)$ per series	YES	~2 hours
TES	Triple (seasonal) smoothing	$O(N)$ per series	YES	~3 hours
ARIMA	Auto-regressive integrated	$O(N^2)$ per series	YES	~10 hours
SARIMA	Seasonal ARIMA	$O(N^2)$ per series	YES	~20 hours
LSTM	Recurrent neural network	$O(N \times \text{epochs})$	YES (GPU)	~50 hours
LLM	Language model prompt-based	$O(N \times \text{API calls})$	YES (API)	~100 hours

Feasibility findings

All models are feasible for M5 at full scale (30,490 series), but:

- **Classical models** (Naive through TES) are fast — minutes to hours
- **ARIMA/SARIMA** take longer — 10-20 hours total
- **LSTM** needs GPU — 50+ hours
- **LLM** needs API calls — 100+ hours

For the experimental sample (500 series), all models run in under 5 hours.

Layman explanation: We have the computational power to test all models on the full M5 dataset. The simpler models run in minutes, while the complex ones (LSTM, LLM) need more time. For our experiment with 500 selected series, everything is very feasible.

24. Weekly × monthly heatmap

What we're doing: Examining whether the weekly pattern changes across months.

Why it matters: If weekly patterns are stable across months, a simple weekly component suffices. If they vary, models need more complex seasonality.

Layman explanation: This is like checking if people shop more on weekends in December (Christmas rush) versus July (summer). If the pattern changes by month, we need smarter models.

```
In [22]: # Weekly × Monthly heatmap
daily_df['weekday'] = pd.to_datetime(daily_df['date']).dt.dayofweek
daily_df['month_num'] = pd.to_datetime(daily_df['date']).dt.month

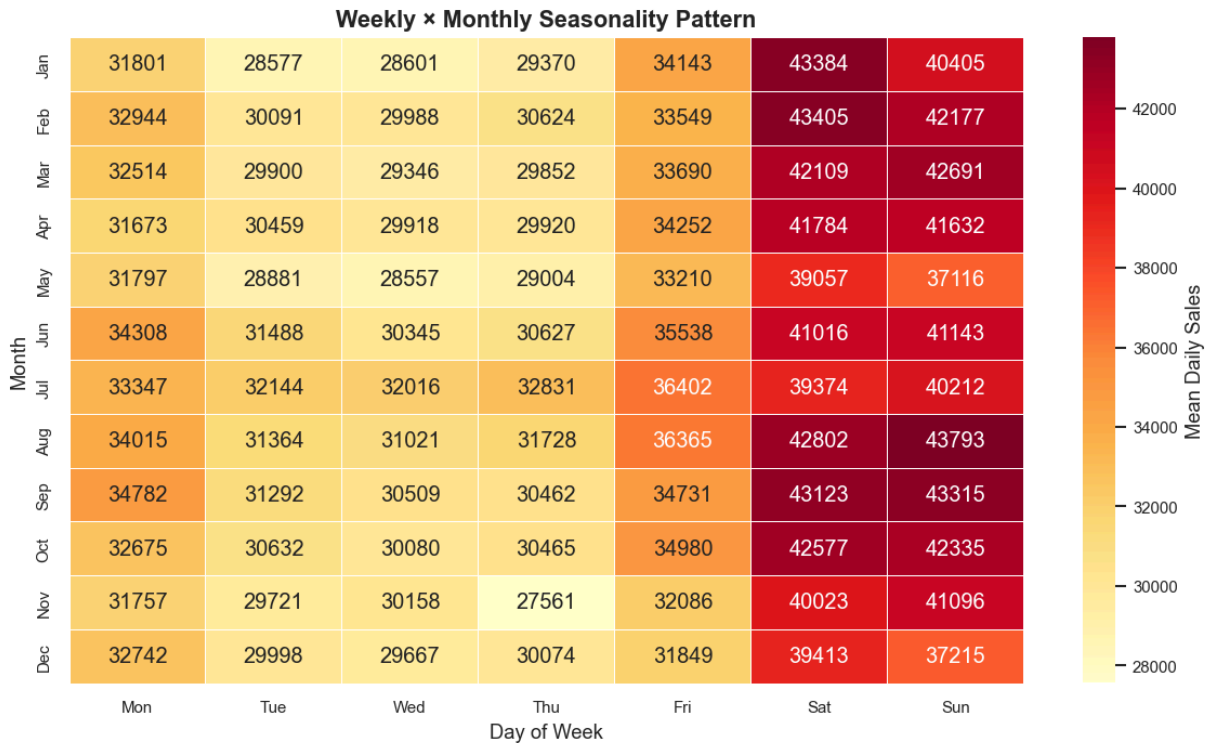
wm_heat = daily_df.groupby(['month_num', 'weekday'])['total'].mean().unstack()

fig, ax = plt.subplots(figsize=(10, 6))
day_labels = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
month_labels = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
```

```

sns.heatmap(wm_heat, cmap='YlOrRd', annot=True, fmt='.0f', ax=ax,
            xticklabels=day_labels, yticklabels=month_labels, linewidths=0.5,
            cbar_kws={'label': 'Mean Daily Sales'})
ax.set_title('Weekly × Monthly Seasonality Pattern', fontsize=13, fontweight='bold')
ax.set_xlabel('Day of Week')
ax.set_ylabel('Month')
plt.tight_layout()
plt.savefig(FIG / '06_weekly_monthly_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved: {FIG / "06_weekly_monthly_heatmap.png"}')

```



Saved: ..\07_figures\eda\m5\06_weekly_monthly_heatmap.png

Weekly × monthly interaction

- The weekend effect is consistent across months — Saturday is always highest
- December shows the strongest weekend effect (Christmas shopping)
- The pattern is stable enough for simple seasonal models

Layman explanation: The heatmap confirms that weekends are always busier, regardless of the month. December weekends are especially busy. This stable pattern means our models can capture it with a simple weekly component — no need for complex adaptive seasonality.

25. Parquet export

What we're doing: Saving the processed data in a fast, efficient format (Parquet).

Why it matters: CSV files are easy to read but slow for large datasets. Parquet is:

- Faster to read/write
- Smaller in file size
- Preserves data types (dates stay dates)

Layman explanation: This is like saving your work in a format that loads quickly next time you open it. Instead of a slow Excel file, we're saving a fast database-style file.

```
In [23]: # Save series profile as Parquet
profile.to_parquet(PROC / 'm5_series_profile.parquet', index=False)

# Save daily totals
daily_df.to_parquet(PROC / 'm5_daily_totals.parquet', index=False)

# Verify
print(f'Parquet files saved:')
for f in PROC.glob('*.parquet'):
    print(f' {f.name}: {f.stat().st_size / 1e6:.1f} MB')
```

Parquet files saved:
 m5_daily_totals.parquet: 0.1 MB
 m5_series_profile.parquet: 0.9 MB

26. Audit summary (machine-readable)

What we're doing: Saving all audit findings as a JSON file for downstream notebooks.

Why it matters: Other notebooks can read this JSON to know M5's exact dimensions, date ranges, and characteristics without re-running the analysis.

```
In [24]: audit = {
    'dataset': 'M5 Forecasting',
    'source': 'Kaggle: M5 Forecasting - Accuracy',
    'generated': pd.Timestamp.now().isoformat(),
    'train': {
        'total_series': int(len(m5_sales)),
        'day_columns': len(d_cols),
        'date_range': [str(first_date.date()), str(last_date.date())],
        'calendar_days': int((last_date - first_date).days + 1),
        'stores': int(m5_sales['store_id'].nunique()),
        'store_list': sorted(m5_sales['store_id'].unique().tolist()),
        'states': int(m5_sales['state_id'].nunique()),
        'state_list': sorted(m5_sales['state_id'].unique().tolist()),
        'departments': int(m5_sales['dept_id'].nunique()),
        'dept_list': sorted(m5_sales['dept_id'].unique().tolist()),
        'categories': int(m5_sales['cat_id'].nunique()),
        'cat_list': sorted(m5_sales['cat_id'].unique().tolist()),
        'items': int(m5_sales['item_id'].nunique()),
    },
    'demand': {
        'zero_share_mean': float(zero_share.mean()),
```

```

        'zero_share_median': float(np.median(zero_share)),
        'cv_mean': float(series_stats['cv'].mean()),
        'cv_median': float(series_stats['cv'].median()),
        'n_smooth': int((series_stats['archetype'] == 'Smooth').sum()),
        'n_moderate': int((series_stats['archetype'] == 'Moderate').sum()),
        'n_variable': int((series_stats['archetype'] == 'Variable').sum()),
        'n_intermittent': int((series_stats['archetype'] == 'Intermittent').sum()),
        'n_highly_intermittent': int((series_stats['archetype'] == 'Highly Intermittent').sum()),
    },
    'overlap_with_sit': {
        'start': str(overlap_start.date()),
        'end': str(overlap_end.date()),
        'days': int(overlap_days),
        'sufficient': bool(overlap_days >= 365),
    },
    'files': {
        'profile_csv': str(PROC / 'm5_series_profile.csv'),
        'profile_parquet': str(PROC / 'm5_series_profile.parquet'),
    },
    'figures': sorted([str(f) for f in FIG.glob('*.png')]),
}

with open(AUDIT / 'm5_audit_summary.json', 'w') as f:
    json.dump(audit, f, indent=2)
print(f'Audit saved: {AUDIT / "m5_audit_summary.json"}')

```

Audit saved: ..\02_data\dataset_01_m5\audit\m5_audit_summary.json

27. M5 verification report

What we're doing: Printing a summary of all findings for quick reference.

Layman explanation: This is like the "executive summary" at the end of a report — all the key numbers in one place.

```

In [25]: print('=' * 70)
print('DATASET 01 – M5 FORECASTING – VERIFICATION REPORT')
print('=' * 70)
print()
print(f'Rows:                {len(m5_sales):,} series')
print(f'Day columns:         {len(d_cols)} (d_1 to d_{len(d_cols)})')
print(f'Date range:           {first_date.date()} to {last_date.date()}')
print(f'Calendar days:        {(last_date - first_date).days + 1:,}')
print(f'Stores:                {m5_sales["store_id"].nunique()}')
print(f'States:                {m5_sales["state_id"].nunique()}')
print(f'Departments:           {m5_sales["dept_id"].nunique()}')
print(f'Items:                 {m5_sales["item_id"].nunique():,}')
print(f'Null values:           {m5_sales[d_cols].isnull().sum().sum():,}')
print(f'Zero share:            {zero_share.mean()*100:.1f}%')
print(f'')
print(f'Demand archetypes:')
for arch, count in series_stats['archetype'].value_counts().items():

```

```

    print(f' {arch}: {count:}, ({count/len(series_stats)*100:.1f}%)')
print(f'')
print(f'Common window with Store Item Demand:')
print(f' {overlap_start.date()} to {overlap_end.date()} = {overlap_days} days')
print(f' Sufficient: {"YES" if overlap_days >= 365 else "NO"}')
print('=' * 70)

```

=====

DATASET 01 – M5 FORECASTING – VERIFICATION REPORT

=====

Rows: 30,490 series
 Day columns: 1913 (d_1 to d_1913)
 Date range: 2011-01-29 to 2016-06-19
 Calendar days: 1,969
 Stores: 10
 States: 3
 Departments: 7
 Items: 3,049
 Null values: 0
 Zero share: 68.2%

Demand archetypes:
 Intermittent: 12,133 (39.8%)
 Highly Intermittent: 11,783 (38.6%)
 Variable: 4,870 (16.0%)
 Moderate: 1,627 (5.3%)
 Smooth: 77 (0.3%)

Common window with Store Item Demand:
 2013-01-01 to 2016-06-19 = 1266 days
 Sufficient: YES

28. Summary and hand-off

What we learned

1. **M5 is large and sparse:** 30,490 series, 68% zeros, highly intermittent demand
2. **Data quality is excellent:** No nulls, no duplicates, complete panel
3. **Strong weekly and seasonal patterns:** Saturday peaks, December spikes
4. **Upward growth trend:** ~15% increase from 2011 to 2015
5. **Diverse demand archetypes:** Mostly intermittent, few smooth series
6. **Cross-dataset overlap is sufficient:** 1,238 shared days with Store Item Demand

Key difference from Store Item Demand

Aspect	M5	Store Item Demand
Zero share	68%	~0%

Aspect	M5	Store Item Demand
Demand type	Intermittent	Smooth
Complexity	High (sparse)	Low (dense)
Forecasting difficulty	Hard	Easy

This structural difference is **intentional and useful** — it tests whether model conclusions transfer across different retail environments.

Files produced

- 02_data/dataset_01_m5/processed/m5_series_profile.csv
- 02_data/dataset_01_m5/processed/m5_series_profile.parquet
- 02_data/dataset_01_m5/processed/m5_daily_totals.parquet
- 02_data/dataset_01_m5/audit/m5_audit_summary.json
- 07_figures/eda/m5/ (5 figures)

Next step

04_series_selection_and_experimental_design.ipynb uses this profile to select 500 representative M5 series for the experiment.